

Katana Pay — Gateway Integration Kit

Hosted-checkout UPI pay-ins for any platform · Base URL `https://katanapay.co` · API v1

What you'll do: your server creates a **signed order** → you redirect the customer to the **Katana Pay hosted payment page** → the customer pays by UPI → Katana **POSTs a signed status callback** to your server *and* redirects the customer back to your `return_url` . You verify every signature with the **Key + Salt** issued to your merchant account.

Contents

- 1. Credentials (Key + Salt)
- 2. Endpoints
- 3. Create an order
- 4. Sign the request
- 5. Redirect to the payment page
- 6. Receive the status callback
- 7. Status enquiry (and long-poll)
- 8. Status & response codes
- 9. Traceability — following one payment end to end
- 10. Sample code (PHP / Node / Python)
- 11. Test mode
- 12. Go-live checklist

1. Credentials (Key + Salt)

Every merchant account gets its **own dedicated Key** and **Salt**, issued from the Katana dashboard (**Integration** → **Generate**, or ask your Katana account manager). They are not shared between merchants, and every order and callback is bound to the pair that signed it.

Test and live pairs

Pair	Key starts with	Orders it creates

Test	<code>mk_test_</code>	Test orders: they pay a sandbox UPI ID, never move real money and never appear in live totals, statements or settlement (§11).
Live	<code>mk_live_</code>	Real payments.

The Key you sign with decides the mode; there is no mode field to send. Keys issued before test mode (`mk_` followed by hex) are live and keep working unchanged.

Field	Meaning
Key	Public-ish identifier (<code>mk_test_...</code> / <code>mk_live_...</code>) sent with every request. Identifies the merchant and the mode.
Salt	Secret. Shown only once at issue. Used to sign your requests and to verify our callbacks. Never expose it in client-side code.
Scheme	Signing algorithm for your account: <code>HMAC_SHA256</code> (recommended) or the legacy SHA-512 format (§4).

Keep each Salt safe. If lost, regenerate that pair — which **invalidates only that pair** (the other mode is untouched), so update your server immediately. One active Key per mode.

2. Endpoints

Purpose	Method	URL
Create order (server-to-server)	POST	<code>https://katanapay.co/api/v1/katana-pay/order</code>
Hosted payment page (browser)	GET	<code>https://katanapay.co/pay/{order_id}</code>
Status enquiry	GET	<code>https://katanapay.co/api/pay-status/{order_id}</code>

Your Key + Salt is the only credential on these endpoints. Treat Salt disclosure as a full compromise of your integration: regenerate immediately and reconcile every order created in the interim.

3. Create an order

POST `/api/v1/katana-pay/order` with a JSON body (form-encoded is also accepted). **Idempotent on `txnid`** — re-sending the same `txnid` returns the existing order rather than creating a second one, so a retry

after a network timeout is always safe. A `txnid` is unique per mode: the same reference sent with a test Key and a live Key creates two separate orders.

Request fields

Field	Req	Description
<code>key</code>	yes	Your Key.
<code>txnid</code>	yes	Your unique order reference (≤ 60 chars). This is your handle on the payment forever — see §9.
<code>amount</code>	yes	Amount in rupees, e.g. <code>"100"</code> or <code>"100.50"</code> .
<code>hash</code>	yes	Signature over the order (see §4).
<code>productinfo</code>	no	Order description (part of the signature).
<code>firstname</code>	no	Customer name (part of the legacy SHA-512 signature).
<code>email</code>	no	Customer email (part of the signature).
<code>phone</code>	no	Customer phone.
<code>return_url</code>	no	Browser redirect target after payment. Overrides your saved default.
<code>notify_url</code>	no	Server-to-server status-callback target. Overrides your saved webhook URL.
<code>currency</code>	no	Defaults to <code>INR</code> .
<code>mode</code>	no	<code>QR</code> or <code>INTENT</code> .

`return_url` and `notify_url` must be **http(s)** URLs. Other schemes are rejected.

Response (HTTP 201 new / 200 reused)

```
{
  "verified": true,
  "merchant": "UK-108",
  "livemode": true,                               // false = created with a test Key
  "order": { "id": "8d20c0b6-...", "order_id": "ORDER-1001", "amount": 100, "status": "PENDING",
  ... },
  "pay_url": "https://katanapay.co/pay/8d20c0b6-...", // redirect the customer here
  "deeplinks": { "upi": "upi://pay?...", ... },
  "upi_intent": "upi://pay?...",
  "qr_payload": "upi://pay?..."
}
```

Take `pay_url` and send the browser there. If you are building your own payment screen instead, use `qr_payload` (render as a QR) or `upi_intent` (an app deeplink).

Error	Meaning
401 invalid key	Unknown Key.
401 signature mismatch	Hash didn't match — check your signing string.
403	Account blocked.
400 invalid amount	Amount missing / not positive.

4. Sign the request

Compute `hash` with the **scheme configured on your account** (shown next to your Key in the dashboard), then send it as the `hash` field. You never send the scheme name — Katana already knows which one your account uses.

HMAC_SHA256 (recommended)

```
message = txnid + "|" + amount + "|" + productinfo + "|" + email
hash     = HMAC_SHA256( key = KEY + SALT , message )      // lowercase hex
```

Legacy SHA-512 format

Used by accounts migrated from an older checkout. Pipe-joined, with five user-defined fields and five reserved blanks before the salt:

```
seq = KEY|txnid|amount|productinfo|firstname|email|1111111111|SALT // 5 udf + 5 reserved
blanks
hash = SHA512(seq) // lowercase hex
```

Introduction

Send the customer's browser to the `pay_url` from the response. The hosted page shows a UPI QR plus app buttons and live status. When the payment is confirmed it automatically redirects back to your `return_url` with query params:

https://your-site.com/return?order_id=ORDER-1001&status=SUCCESS&rrn=123456789012

Always re-confirm server-side (callback or status enquiry) before fulfilling — the browser redirect is **informational only** and can be forged by anyone who can type a URL.

On every terminal status Katana **POSTs** a JSON body to your `notify_url` (or your saved webhook URL). Reply `HTTP 200`. Delivery is queued and **retried with backoff** until you do, and every attempt is recorded — so a callback is never silently dropped.

```
{
  "PAY_ID": "...", "ORDER_ID": "ORDER-1001", "TXN_ID": "...",
  "AMOUNT": "100", "CURRENCY_CODE": "356",
  "STATUS": "Captured", "RESPONSE_CODE": "000",
  "RRN": "123456789012", "RESPONSE_DATE_TIME": "2026-07-01T...Z",
  "HASH": "716346F5...EC21"
}
```

Verify the HASH (with your Salt)

1. Take every field **except** `HASH` .
2. Sort the keys **ascending**; join as `KEY=value` with `~` between pairs.
3. Append your `SALT` directly to the end of that string.
4. `SHA256` it $\rightarrow$ hex $\rightarrow$ **UPPERCASE**. It must equal `HASH` .

Paid = STATUS is Captured and RESPONSE_CODE is 000 .

Test callbacks. A callback for a test order also carries "LIVEMODE": "false" (included in the HASH like any other field) and is signed with the **test Salt**. Live callbacks omit the field. Verify with the Salt of the matching mode, and never fulfil a real order from a LIVEMODE=false callback.

Treat callbacks as idempotent. The same ORDER_ID may arrive more than once (a retry, or a status that settles twice). Key your handler on ORDER_ID and make a second delivery a no-op.

7. Status enquiry (and long-poll)

At any time: GET /api/pay-status/{order_id}

```
{ "order_id": "ORDER-1001", "amount": 100, "currency_code": "INR",  
  "status": "SUCCESS", "terminal": true, "rrn": "123456789012",  
  "mode": "QR", "livemode": true, "return_url": "..." }
```

terminal: true means the order has reached a final state and will not change again.

Long-poll: ?wait=1

Add ?wait=1 and the request is **held open until the order becomes terminal** (up to ~25s), instead of returning the current state immediately. This lets your own page flip to "Payment received" within about half a second of the money landing, without polling in a tight loop.

```
GET https://katanapay.co/api/pay-status/8d20c0b6-...?wait=1
```

Use plain enquiry (no wait) for reconciliation sweeps, and wait=1 only where a person is watching a screen. Either way, enquiry is the **fallback if a callback was missed** — it always reflects the truth.

8. Status & response codes

STATUS	RESPONSE_CODE	Terminal	Meaning
Captured	000	yes	Paid — fulfil the order.
Pending	005	no	Awaiting payment.

Failed	004	yes	Declined / failed.
Expired	003	yes	Payment request lapsed.

9. Traceability — following one payment end to end

Every pay-in carries four identifiers. Store all of them against your order; each answers a different question.

Identifier	Who creates it	Use it to
<code>txnid / ORDER_ID</code>	You , on create	Your own reference. Idempotency key on create, and the key you match callbacks on.
<code>order.id</code> (UUID)	Katana	Address the hosted page (<code>/pay/{id}</code>) and the status endpoint.
<code>PAY_ID</code>	Katana	Quote to support — it identifies the payment inside Katana.
<code>RRN / UTR</code>	The UPI network	The 12-digit bank reference. This is what a bank statement shows, so it is the identifier that settles disputes.

The lifecycle

```

create order      → status PENDING      (you hold txnid + order.id)
customer pays by UPI → credit confirmed
                  → status SUCCESS, RRN filled in
                  → signed callback POSTed to your notify_url (retried until 200)
                  → customer redirected to your return_url

```

A payment that never completes ends **Expired**; one that fails ends **Failed**. Both are terminal and both fire a callback, so your side never has to guess.

Reconciling a day

- For any order, `GET /api/pay-status/{order_id}` is the authoritative answer at that moment.
- Match your ledger to the bank's on **RRN** — it is the only identifier both sides share.
- In the dashboard, **Transactions** lists every order for your account with its status, RRN and timestamps, and exports the same view as CSV for the period you filter to.
- If a callback never arrived, enquiry will still show the true status — a missed callback never means a lost payment.

Reconciliation rule of thumb: your system should be able to answer "what happened to ORDER-1001 ?" from your own database alone. That is what storing the four identifiers above buys you.

10. Sample code

PHP — create order

```
$key="mk_xxx"; $salt="your_salt";
$o = ["txnid"=>"ORDER-1001","amount"=>"100","productinfo"=>"Order 1001","email"=>"d@e.com"];
$msg = $o["txnid"]."|" . $o["amount"]."|" . $o["productinfo"]."|" . $o["email"];
$o["hash"] = hash_hmac("sha256", $msg, $key.$salt);
$o["key"]=$key; $o["return_url"]="https://you.com/return";
$o["notify_url"]="https://you.com/callback";
$ch=curl_init("https://katanapay.co/api/v1/katana-pay/order");
curl_setopt_array($ch, [CURLOPT_POST=>1,CURLOPT_RETURNTRANSFER=>1,
    CURLOPT_HTTPHEADER=>["Content-Type:
application/json"],CURLOPT_POSTFIELDS=>json_encode($o)]);
$res=json_decode(curl_exec($ch),true);
header("Location: ".$res["pay_url"]); // redirect customer
```

Node.js — create order

```
import crypto from "crypto";
const key="mk_xxx", salt="your_salt";
const o={ txnid:"ORDER-1001", amount:"100", productinfo:"Order 1001", email:"d@e.com" };
const msg=[o.txnid,o.amount,o.productinfo,o.email].join("|");
o.hash=crypto.createHmac("sha256",key+salt).update(msg).digest("hex");
const r=await fetch("https://katanapay.co/api/v1/katana-pay/order",{method:"POST",
    headers:{"Content-Type":"application/json"},

    body:JSON.stringify({...o,key,return_url:"https://you.com/return",notify_url:"https://you.com/
callback"}}));
const data=await r.json(); // redirect customer to data.pay_url
```

Python — create order

```
import hmac,hashlib,requests
key,salt="mk_xxx","your_salt"
o={"txnid":"ORDER-1001","amount":"100","productinfo":"Order 1001","email":"d@e.com"}
msg="|".join([o["txnid"],o["amount"],o["productinfo"],o["email"]])
o["hash"]=hmac.new((key+salt).encode(),msg.encode(),hashlib.sha256).hexdigest()
o.update(key=key, return_url="https://you.com/return", notify_url="https://you.com/callback")
data=requests.post("https://katanapay.co/api/v1/katana-pay/order",json=o).json()
# redirect customer to data["pay_url"]
```

Verify a callback (Node.js)

```
function verify(body, salt){
  const got=body.HASH; const f={...body}; delete f.HASH;
  const str=Object.keys(f).sort().map(k=>`${k}=${f[k]==null?"":f[k]}`).join("~")+salt;
  const exp=crypto.createHash("sha256").update(str,"utf8").digest("hex").toUpperCase();
  return exp===got;    // true → trust it; then check STATUS==="Captured"
}
```

Verify a callback (PHP)

```
function verify($body,$salt){
  $got=$body["HASH"]; unset($body["HASH"]); ksort($body);
  $parts=[]; foreach($body as $k=>$v){$parts[]=$k."($v??)";}
  $str=implode("~",$parts).$salt;
  return strtoupper(hash("sha256",$str))===$got;
}
```

11. Test mode

Integrate with the **test pair** first. Test orders behave exactly like live ones (hosted page, callbacks, status enquiry) except that they pay a sandbox UPI ID, the hosted page shows a **Test payment** label, and the paise of the amount force the outcome so every branch can be tested without a real payer:

Send amount	Order becomes	Use it to test
100.99	SUCCESS after ~8 s	Happy path, callback verification, fulfilment
100.13	FAILED	Decline handling
100.11	EXPIRED	Expiry handling

100.00

stays PENDING , expires after 15 min

Long-poll and your own re-check

These rules apply **only** to test orders. A live order changes state only on a real confirmed credit, and an amount ending in .99 behaves exactly like any other. Real bank credits never confirm a test order.

In the dashboard, the **Test / Live** switch in the header shows test orders; an orange banner stays on while you are viewing test data. Live totals, statements and settlement never include test orders.

12. Go-live checklist

- Complete a test integration with the **test Key + Salt**: success, failure and expiry callbacks all verify (§11).
 - Generate your **live Key + Salt**, store the Salt securely (server-side only) and swap both on your server.
 - Set your default **return_url** and **webhook URL** in the dashboard (or pass them per order).
 - Store `txnid` , `order.id` , `PAY_ID` and `RRN` against your order (§9).
 - Make a small live payment; confirm you receive the callback and the **HASH verifies** with the live Salt.
 - Handle callbacks **idempotently** — the same `ORDER_ID` may arrive more than once.
 - Make create-order **retry-safe** by always reusing the same `txnid` .
 - Always confirm **server-side** before fulfilling — never trust the browser redirect alone.
-

Katana Pay · integration support: [contact your Katana account manager](#).